Trabalho #2 - Guia de execução e prazos

Disciplina: Programação Web - DCE701 - UNIFAL-MG - 2026/2

Projeto: Reading Tracker Web Application (aplicação para acompanhar leituras)

Base: enunciado Reading_Tracker_Web_Application.pdf, atualizado em 16/10/2026. Guia elaborado em 19/10/2026.

Este guia resume o enunciado para ajudar no planejamento. As datas e exigências abaixo vêm do documento da disciplina; o cronograma de desenvolvimento ao final é uma sugestão. Consulte o Moodle para eventuais atualizações.

1. O que precisamos construir

Uma aplicação web que substitua o documento usado por uma pessoa para registrar livros lidos e livros que pretende ler. O sistema deve organizar **autores, livros e leituras**, permitir cadastrar e corrigir informações e manter os dados salvos.

Um **livro** representa a obra. Uma **leitura** representa uma ocasião em que a pessoa leu ou pretende ler essa obra. Assim, o mesmo livro pode ter várias leituras, sem duplicar seu cadastro.

Idioma da aplicação: a interface e os valores padronizados devem estar em **inglês dos Estados Unidos**. Títulos de livros e nomes de autores permanecem no idioma original. Este guia está em português apenas para facilitar nossos estudos. Centralize os textos da interface para facilitar uma tradução futura.

2. Prazos oficiais e pontuação

Entrega	Data e horário limite	O que entregar	Valor
1 - Frontend	**20/11/2026, às 23h59**	Interface completa, com dados no navegador	5,0 pontos
2 - Full stack	**11/12/2026, às 23h59**	Frontend integrado à API Spring Boot e armazenamento persistente	5,0 pontos

Fuso horário: Brasília. O trabalho vale 10 pontos e tem peso 0,3 na nota final da disciplina, isto é, 30%.

- Pode ser individual ou em equipe de **até cinco estudantes**; as exigências são iguais.
- Os integrantes devem constar no README da primeira entrega e **não podem mudar na segunda**.
- As duas entregas devem estar disponíveis em repositório **público no GitHub**.
- Até 24 horas de atraso: o enunciado prevê penalidade de **20% sobre a pontuação total do trabalho**. Commits e e-mails enviados depois dessa janela são desconsiderados; uma entrega sem submissão válida nessa janela recebe zero.
- Tanto o e-mail quanto o commit avaliado precisam chegar dentro do prazo para a entrega ser pontual.

3. Linguagens, tecnologias e ferramentas

Parte	Exigência do enunciado	Para que serve
Estrutura das páginas	**HTML puro**	Formulários, tabelas, títulos e conteúdo

Parte	Exigência do enunciado	Para que serve
Aparência	**CSS puro**	Layout, cores e adaptação a computador e celular
Comportamento	**JavaScript puro**	Eventos, validações, manipulação do DOM e dados
Comunicação	**Fetch API e JSON**	Carregar o arquivo inicial e chamar a API
Dados da entrega 1	**localStorage**	Preservar os dados ao recarregar a página
Backend da entrega 2	**Java 21 ou superior e Spring Boot 4.x**	Implementar a API REST e as regras no servidor
Gerenciamento do backend	**Maven ou Gradle**	Executar e organizar as dependências
Persistência da entrega 2	Livre, desde que sobreviva ao reinício	Armazenar os dados permanentemente

Caminho recomendado pelo próprio enunciado: Spring Web + Spring Data JPA + Validation + **H2 em arquivo**. O H2 em memória não atende à exigência de persistência. Com JPA, não é necessário escrever todo o SQL manualmente.

Ferramentas de trabalho: podemos usar VS Code para editar os arquivos e Git/GitHub para versionar e entregar. Na primeira etapa, o enunciado sugere Live Server para servir o frontend. Na segunda, o próprio Spring Boot serve as páginas. MAMP e WordPress não são exigidos neste trabalho; o backend obrigatório é Spring Boot.

Restrições importantes

- **Não usar:** React, Vue, Angular, Svelte, jQuery, Bootstrap, Tailwind, TypeScript, Vite, Webpack ou ferramentas de build/bundlers e pacotes npm para construir o frontend.
- **Não renderizar HTML no servidor** com Thymeleaf ou JSP.
- São permitidos Chart.js ou biblioteca pequena semelhante para gráficos, Google Fonts e fontes de ícones. Gráficos são opcionais.
- No backend, são permitidos Spring Web, Spring Data JPA, Bean Validation, Lombok, o driver do banco escolhido, DevTools e o starter de testes do Spring Boot; o enunciado também permite o console H2.
- Qualquer outra biblioteca precisa de aprovação prévia do professor por e-mail, registrada no README. Spring Security é expressamente permitido para o exercício opcional de login.

4. Checklist obrigatório - nas duas entregas

ID	O que implementar	Critério prático
FR-01	Gerenciar autores	Criar, consultar, editar e excluir. Impedir exclusão de autor vinculado a livros, informando quais são.
FR-02	Gerenciar livros	Criar, consultar, editar e excluir. Selecionar autores e temas existentes; permitir criar tema no formulário.
FR-03	Gerenciar leituras	Adicionar, editar e excluir leituras. Permitir cadastrar livro e sua primeira leitura em um único salvamento. Ao excluir livro, confirmar e informar quantas leituras serão excluídas junto.

ID	O que implementar	Critério prático
FR-04	Alterar status	Trabalhar com Want to read, Reading, Paused, Read e Abandoned, respeitando as regras dos campos.
FR-05	Mostrar histórico de leituras	Exibir título, autores, status, ano lido e nota; permitir ordenar por cada coluna. Mostrar ausências como -, depois dos valores preenchidos. Ordem inicial: ano lido decrescente e, no empate, ID crescente.
FR-06	Validar os dados	Aplicar todas as regras da seção 4 do enunciado; mostrar mensagens junto aos campos e não salvar dados inválidos.
FR-07	Carregar dados iniciais e restaurar	Importar seed.json apenas no primeiro uso; preservar alterações nas próximas visitas; oferecer Reset data com confirmação.

Dados e validações que não podemos esquecer

- **Autores:** nome obrigatório e único sem diferenciar maiúsculas de minúsculas; nacionalidade opcional escolhida da lista fixa de países; nascimento não pode estar no futuro; falecimento deve ser posterior ao nascimento.
- **Livros:** título obrigatório; autores opcionais e selecionados de cadastros existentes; ano de publicação opcional, nunca futuro, sem ano zero; aceitar anos negativos para obras anteriores a Cristo. Forma, modo de expressão e conteúdo usam as listas padronizadas da seção 4. Temas não podem se repetir, e seus nomes são únicos sem diferenciar maiúsculas de minúsculas.
- **Leituras:** livro e status obrigatórios. yearRead é obrigatório em Read, opcional em Abandoned e vazio nos demais estados. Não pode ser futuro nem anterior ao ano de publicação do livro.
- **Notas:** opcionais, de 0 a 10 em intervalos de 0,5; permitidas somente em Read ou Abandoned.
- **Prioridade:** opcional, com valores Low, Medium e High; permitida somente em Want to read.
- Ao sair de Want to read, apagar a prioridade e manter recomendação e observações. Se a mudança invalidar ano lido ou nota, pedir confirmação antes de apagar esses valores.
- Para registrar uma releitura, criar outra leitura, preservando a anterior.
- Países vêm do arquivo inicial e não podem ser criados, editados ou excluídos pela aplicação.
- Tratar campos ausentes sem mostrar null, undefined ou NaN na interface. Usar null e listas vazias no armazenamento conforme o formato do seed.

As listas completas de formas literárias e demais valores controlados estão na seção 4 do PDF e devem ser usadas com grafia consistente em inglês.

5. Entrega 1 - o que precisamos fazer até 20/11

[ ] Criar as telas e implementar FR-01 a FR-07 com HTML, CSS e JavaScript puros.

[ ] Colocar seed.json no frontend, por exemplo em data/seed.json.

[ ] Carregar o seed com fetch no primeiro uso e gravá-lo no localStorage.

[ ] Nas visitas seguintes, usar os dados salvos; não sobrescrever alterações com o seed.

[ ] Gerar IDs inteiros novos a partir do maior ID existente em cada coleção.

[ ] Implementar a restauração apenas das chaves de armazenamento da aplicação, com confirmação, recarregando o seed.

[ ] Separar HTML, CSS e JavaScript; dividir JavaScript por acesso a dados, renderização e validação.

[ ] Concentrar acesso a dados em um arquivo como store.js, com funções assíncronas chamadas por await, preparando a troca por API na entrega 2.

[ ] Usar os mesmos formatos de dados do seed; erros de validação devem ter estrutura compatível com a futura resposta HTTP 400 da API.

[ ] Usar addEventListener; não usar onclick, style inline nem blocos <style> no HTML.

[ ] Usar HTML semântico e verificar funcionamento em notebook e celular.

[ ] Executar por servidor HTTP local, como Live Server; não abrir apenas por file://.

[ ] Preparar README, publicar código e tag e enviar e-mail com link e autoavaliação de 0 a 5.

Podemos usar scripts separados com defer ou módulos ES com import/export; ambos são aceitos, mas módulos ES são recomendados no enunciado. Não há vídeo obrigatório na entrega 1.

6. Entrega 2 - o que precisamos fazer até 11/12

[ ] Criar o projeto Java 21+ com Spring Boot 4.x.

[ ] Separar **controllers** (HTTP), **services** (regras) e **repositories** (dados). Não colocar regras de negócio nos controllers.

[ ] Implementar a API REST em JSON para autores, livros, leituras, temas, países e restauração.

[ ] Revalidar as regras no servidor; responder com códigos HTTP adequados e mensagens que o frontend consiga mostrar junto aos campos.

[ ] Manter dados após reiniciar a aplicação; usar H2 em arquivo como opção recomendada.

[ ] Importar seed.json no primeiro início com banco vazio; não reimportar sobre dados existentes.

[ ] Deixar o banco gerar os IDs e mapear os IDs do seed para os registros salvos. Importar na ordem: países, temas, autores, livros e leituras.

[ ] Reutilizar a rotina de importação em POST /api/reset; não depender de IDs fixos após restauração.

[ ] Alterar store.js para chamar a API com fetch, mantendo todas as funcionalidades da entrega 1. Não usar localStorage para os dados da aplicação nesta etapa.

[ ] Copiar o frontend para src/main/resources/static e usar URLs relativas, como /api/books.

[ ] Garantir que toda a aplicação execute com um único comando; no Windows com Maven: mvnw.cmd spring-boot:run, ou equivalente com Gradle.

[ ] Versionar o wrapper de build. Colocar o seed em src/main/resources/seed.json; não versionar a pasta de dados gerada pelo H2.

[ ] Atualizar README, gravar vídeo, publicar código e tag e enviar o e-mail da segunda entrega.

API mínima prevista no enunciado

Métodos	Caminho	Finalidade
GET, POST	/api/authors	Listar e criar autores

Métodos	Caminho	Finalidade
GET, PUT, DELETE	/api/authors/{id}	Consultar, atualizar e excluir autor
GET, POST	/api/books	Listar e criar livros
GET, PUT, DELETE	/api/books/{id}	Consultar, atualizar e excluir livro e suas leituras
GET, POST	/api/books/{id}/readings	Listar e adicionar leituras de um livro
GET	/api/readings	Listar todas as leituras
PUT, DELETE	/api/readings/{id}	Atualizar e excluir leitura
GET, POST	/api/themes	Listar e criar temas
GET	/api/countries	Listar países
POST	/api/reset	Restaurar os dados iniciais

Usar corretamente 200, 201, 204, 400, 404 e 409. Os erros devem seguir a estrutura da seção 7: status, message e lista errors com field e message. Erros 404 e 409 usam essa estrutura com lista de campos vazia. Os caminhos podem ser renomeados se mantiverem as convenções REST e forem documentados.

7. Como entregar

Enviar cada entrega para **pagliares@bcc.unifal-mg.edu.br** até o respectivo prazo. Em equipe, um integrante envia e coloca os demais em cópia.

Item	Entrega 1	Entrega 2
Link do repositório público GitHub	Obrigatório	Obrigatório
Autoavaliação de 0,0 a 5,0	Obrigatória	Obrigatória
Código-fonte e seed.json	Obrigatórios	Obrigatórios
README com integrantes, nomes e matrículas	Obrigatório	Manter a mesma equipe
Instruções para executar	Obrigatórias	Atualizadas, com versão do Java e comando único
Checklist FR-01 a FR-07	Implementado, parcial ou não implementado	Atualizado
Declaração do uso de IA	Obrigatória, inclusive para declarar que não usou	Atualizada
Lista de endpoints e exemplo de requisição/resposta por recurso principal	-	Obrigatórios para autores, livros e leituras
Vídeo acessível sem login, com link no README e na submissão	-	Obrigatório, **máximo de 15 minutos**

Assunto do e-mail: Reading Tracker - Delivery 1 - [nomes dos estudantes]; na segunda etapa, trocar por Delivery 2.

Marcar o commit a ser avaliado com **delivery-1** ou **delivery-2** e enviar a tag ao GitHub antes do prazo. Sem tag, o professor avalia o último commit enviado. Fazer commits durante o desenvolvimento, com mensagens descritivas e participação da equipe visível.

No vídeo da entrega 2, primeiro demonstrar os requisitos e a persistência após reiniciar a aplicação; depois explicar controllers, services, repositories, configuração de persistência e chamadas da API pelo frontend. Só os primeiros 15 minutos serão avaliados.

Uso de IA: permitido com declaração no README de ferramenta, finalidade e partes afetadas. Cada integrante deve entender o código entregue. O enunciado prevê perda de 40% dos pontos da entrega para quem não conseguir explicar seu código em eventual conversa com o professor. Código adaptado de fontes externas precisa de referência em comentário.

8. Prioridades para a nota

Na entrega 1, FR-01 a FR-07 somam **3,0 pontos**; qualidade do código e da interface vale **1,0**, e README vale **1,0**.

Na entrega 2: API REST **1,5**; persistência **1,0**; integração frontend/API **1,0**; validação e erros no servidor **0,5**; arquitetura em camadas **0,5**; README e vídeo **0,5**.

Se o projeto não executar conforme as instruções do README, a nota fica limitada a **50% dos pontos da entrega**. Bibliotecas proibidas retiram os pontos dos critérios que dependem delas.

FR-08 a FR-18 são **opcionais e não valem pontos**: filtros e busca, visão de lista de desejos, livros por século, autores por nacionalidade, estatísticas, gráficos, Open Library, importação/exportação, login, banco externo com Docker e DTOs/tratamento global de erros. Priorizar o obrigatório antes dessas extensões. Docker só é exigido se fizermos FR-17.

9. Cronograma sugerido de desenvolvimento

Este cronograma é uma proposta de organização, não uma exigência do professor. Ajustar o esforço à experiência e à quantidade de integrantes, mantendo os prazos oficiais.

Período	Objetivo
19 a 25/10	Ler o enunciado, definir equipe, criar repositório e README, obter os arquivos de dados e revisar HTML/CSS/JS.
26/10 a 01/11	Montar telas, formulários e modelo de dados; implementar carga inicial e localStorage.
02 a 08/11	Implementar autores, livros e leituras; validar regras e mudanças de status.
09 a 15/11	Concluir histórico ordenável, restauração, responsividade e testes de todos os FRs obrigatórios.
16 a 19/11	Corrigir problemas, testar a execução em ambiente limpo e finalizar README e autoavaliação.
20/11, até 23h59	**Entrega 1:** publicar commit/tag e enviar e-mail.
21 a 27/11	Criar backend, entidades, banco persistente, importação do seed e endpoints.
28/11 a 04/12	Integrar frontend/API; concluir regras no servidor, erros e restauração.
05 a 10/12	Testar reinício, persistência, instalação e comando único; revisar README e gravar vídeo.

Período	Objetivo
11/12, até 23h59	**Entrega 2:** publicar commit/tag e enviar e-mail com vídeo e autoavaliação.

10. Por onde começar e o que estudar

1. **Resolver o acesso aos materiais iniciais:** precisamos de seed.json, How_Books_Read_Currently_Tracked.pdf e Students_Seed_Data_Readme.pdf. Na consulta ao Moodle de 19/10/2026, esses três links retornaram erro de registro inexistente. Conferir novamente e, se necessário, solicitar os arquivos ao professor. Não substituir silenciosamente o conjunto oficial por dados inventados na entrega.
2. **Definir equipe e criar um repositório público próprio para o Reading Tracker**, com README desde o começo. Um repositório próprio é uma sugestão de organização; o requisito é disponibilizar publicamente as entregas no GitHub.
3. **Estudar para o frontend:** HTML semântico, formulários, tabelas, CSS responsivo/Flexbox, DOM, eventos, funções, arrays, objetos, JSON, validação, localStorage, Promises, async/await, fetch e módulos JavaScript.
4. **Construir primeiro um fluxo completo:** cadastrar autor → cadastrar livro → registrar leitura → recarregar a página e verificar persistência. Depois completar os demais requisitos.
5. **Estudar para o backend:** Java, HTTP/REST, JSON, controllers/services/repositories, Spring Boot, JPA, relacionamentos e Bean Validation. O enunciado indica os vídeos 36 a 44 da playlist da disciplina. Alguns exemplos usam Spring Boot 3: para o trabalho, seguir Spring Boot 4.x e as observações do PDF sobre Jackson 3 e console H2.

Fontes da disciplina

- **Enunciado principal:** Reading_Tracker_Web_Application.pdf, atualização de 16/10/2026, especialmente seções 2, 3, 4, 5, 6, 7, 8, 9, 10 e 11.
- **Slides:** Slides_Reading-Tracker-Semester-Assignment.pdf.
- Curso no Moodle.
- Trabalhos do semestre no Moodle.

Os PDFs citados já estão na pasta Docs da disciplina. Este guia é material de apoio e não substitui o enunciado original.